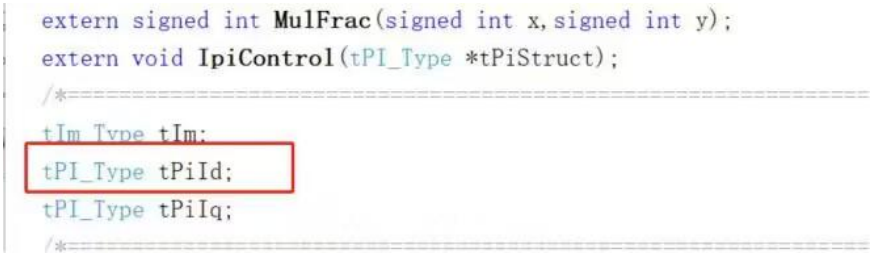
60-FOC 13

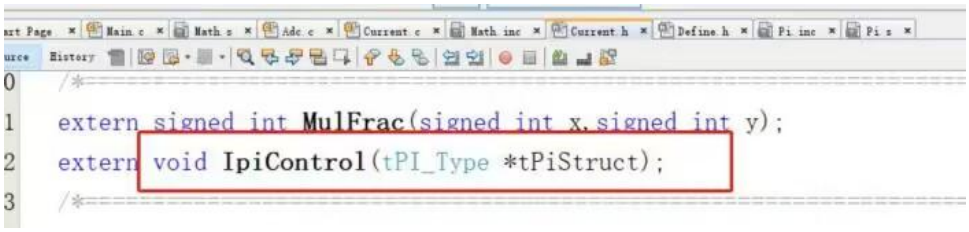
大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习
上节课我讲了定义 PI 结构体变量。



上面就是在汇编和 C 语言中定义结构体类型。



定义变量就是这样定义的。



这两个变量会在这个电流环控制函数中应用。

问：放在函数形参怎么理解？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：形参是在定义函数时指定的参数，用于接收函数调用时传递的参数，形参出现在函数的定义中，用于说明函数的输入和输出。这个形参是指向结构体类型的指针。

```
void CurrentLoop(void)
{
    GetIdq();

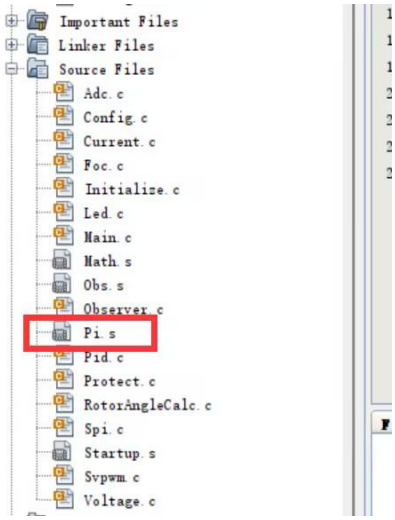
    tIm.qIdGiven = 0;
    tPiId.sErr = tIm.qIdGiven - tIm.qId;
    tPiId.Kp = 8192;
    tPiId.Ki = tPiId.Kp >> 6;
    tPiId.sLimit = 32767;
    IpiControl(&tPiId);
    tUm.qUd = tPiId.sOut;

    tPiIq.sErr = tIm.qIqGiven - tIm.qIq;
    tPiIq.Kp = 8192;
    tPiIq.Ki = tPiId.Kp >> 6;
    tPiIq.sLimit = 32767;
    IpiControl(&tPiIq);
    tUm.qUq = tPiIq.sOut;
}
```

问：r = tIm.qIqGiven - 没有赋初值？

答：初值在启动文件中由 RAM 清零，赋值为 0 了。

上面的内容理解之后，我们来看下对应的函数实现啊。



在 pi.s 中实现。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    .section    .Pi_section,code
/*=============================================================================
;函数输入：w0
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5  ;误差存放在寄存器w5中
    mpy     w4*w5,A  ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1  ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
```

我们现在看下这段代码啊。
首先呢，我们要来计算比例项。

$$U(n)=\boxed{Kp*error(n)}+Ki*\sum_{i=0}^{n}Kp*error(i)$$

```
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5  ;误差存放在寄存器w5中
    mpy     w4*w5,A  ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1  ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
```

这一项的话，也就对应着这 4 句话。前两句分别把 Kp　sErr 放到 w4　w5 寄存器中。然后，使用 mpy 指令来计算乘法，结果存入累加器 A 中。接着呢，取出 Q15 结果，存到 W3 寄存器中。这样的乘法，我们上节课就遇到过。也就是说，此时 w3 寄存器中，就对应比例项的输出。

问：形参和变量一样吗？
答：形参和变量不一样，形参是用来传递参数的。在函数被调用之前，形参不占用内存空间，只有在函数被调用时，形参才会被分配内存空间。而变量在函数被调用之前，是占用内存空间的，全局变量会一直占用固定地址空间的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5  ;误差存放在寄存器w5中
    mpy     w4*w5,A  ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1  ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
    clr     w2  ;清零w2寄存器
```

接着，我们要来看下这段逻辑。这里，其实就是判断一下比例项的输出，是不是超过设定的限幅值。如果超过的话，就让它等于限幅。而限幅值的话，有正的，也有负的，所以需要分别来判断。我们先把内部的 PI 函数讲完，再回来讲电流环。

```
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5  ;误差存放在寄存器w5中
    mpy     w4*w5,A  ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1  ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
```

首先呢，先把限幅值存到 w1 寄存器中。w3 中存储的是比例项输出值，所以就比较这两个值。

```
    mov     [w0+pi_sLimit],w1  ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
```

比较的话，这里用了一条 cpsgt 的指令，我们呢，也来查一下这条指令的含义啊。

CPSEQ	Wb, Wn	比较 (Wb – Wn)，如果相等则跳过	1	1 (2 或 3)	207
CPSGT	Wb, Wn	有符号比较 (Wb – Wn)，如果大于则跳过	1	1 (2 或 3)	211
CPSLT	Wb, Wn	有符号比较 (Wb – Wn)，如果小于则跳过	1	1 (2 或 3)	212

大家看下，这是一条有符号比较的指令，比较的话，其实就是做一个减法来比较。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

语法：	{标号:} CPSGT{.B} Wb, Wn
操作数：	Wb ∈ [W0 ... W15] Wn ∈ [W0 ... W15]
操作：	(Wb) – (Wn) 如果 (Wb) > (Wn)，则跳过
受影响的状态位：	无
指令编码：	1110 | 0110 | 0www | wB00 | 0001 | ssss
描述：	通过执行减法 (Wb) – (Wn)，将 Wb 和 Wn 的内容进行比较，但并不保存计算结果。如果 (Wb) > (Wn)，将放弃执行下一条指令（在当前指令执行过程中取出），而在下一个指令周期内执行一条 NOP 指令。否则将顺序执行下一条指令。

大家来看，如果 wb 大于 wn，则这条指令下面的一条指令就会被跳过。

```
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
```

大家来看，如果说 w3 大于 w1，也就是说比例项输出大于限幅值了。

此时呢，我们就需要强制让比例项的输出等于限幅值，也就是会跳过 goto 指令，而执行下面的指令。好，我们继续分析。

```
    mov     [w0+pi_sLimit],w1 ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1  ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2  ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
```

那么，另一种情况就是，比例项输出没有大于限幅值。这个时候呢，会执行 goto 指令，程序流会跳转到标号_Ipi_j1 处。那么，这里要干嘛呢？我们刚才的比较，是比例项输出小于了正的限幅值，但是，不能保证它不会小于负的限幅值。所以，这里还要再跟负的限幅值比较一下，这样才能确定最终的比例项输出值。

```
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
    clr     w2       ;清零w2寄存器
```

所以，这个时候，先使用一条 neg 指令，把限幅值变为负数，存到 w2 中。

```
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
    clr     w2       ;清零w2寄存器
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

然后，接着呢，就用 cpslt 指令来比较

CPSEQ	Wb, Wn	比较 (Wb – Wn)，如果相等则跳过	1	1 (2 或 3)	207
CPSGT	Wb, Wn	有符号比较 (Wb – Wn)，如果大于则跳过	1	1 (2 或 3)	211
CPSLT	Wb, Wn	有符号比较 (Wb – Wn)，如果小于则跳过	1	1 (2 或 3)	212
CPSNE	Wb, Wn	比较 (Wb – Wn)，如果不等于则跳过	1	1 (2 或 3)	214

注意这里的比较是有符号比较。现在呢，是小于就跳过。

```
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
    clr     w2   ;清零w2寄存器
    cpsgt   w3,w2    ;如果比例项输出w3大于0，跳过下一条语句
```

也就是说，如果比例项输出小于负的限幅值，说明此时呢输出值超过了负的限幅值。那么，就要把比例项输出给限制住，强制赋值负的限幅值给比例项输出。所以，就会执行红色箭头的 mov 指令，把 w2 的值赋值给 w3。

```
    mov     [w0+pi_sLimit],w1 ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1 ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2 ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
    clr     w2   ;清零w2寄存器
    cpsgt   w3,w2    ;如果比例项输出w3大于0，跳过下一条语句
    goto    _Ipi_j3 ;如果比例项输出w3小于等于0，跳转到标号_Ipi_j3执行
    mov     w3,w2    ;比例项输出临时存放于w2
    goto    _Ipi_j4
```

那么，如果比例项输出没有超过负的限幅值，则会执行 goto 指令，跳转到标号_Ipi_j2。

```
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5 ;误差存放在寄存器w5中
    mpy     w4*w5,A ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1 ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
    goto    _Ipi_j1 ;如果w1>=w3,跳转到_Ipi_j1执行
    mov     w1,w3    ;如果比例项输出大于输出限幅值，将输出限幅值作为比例项输出
    goto    _Ipi_j2 ;跳转到_Ipi_j2执行
_Ipi_j1:
    neg     w1,w2    ;w2寄存器存放负的输出限幅值
    cpslt   w3,w2    ;如果比例项的输出小于负的输出限幅值，跳过下一条指令
    goto    _Ipi_j2  ;如果比例项输出大于等于负的输出限幅值,跳转到_Ipi_j2执行
    mov     w2,w3    ;如果比例项输出小于负的限幅值，将负的输出限幅值作为比例项输出
_Ipi_j2:
```

所以呢，代码到这里，就是计算了一个比例项输出，并且对比例项做了一个限幅。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5  ;误差存放在寄存器w5中
```

问：为什么 w0+pi_kp，可以是 pi_kp 不加 w0 吗？基地址是从 W0 开始的吗？

答：不可以，pi_Kp 只是偏移量，不是完整的地址。基地址是通过形参传至 W0 的。

问：基地址可以从 W1 开始的吗？

答：不行，汇编函数默认第一个形参，就是 W0。

问：一般有积分限幅，比列也限的考虑有哪些？

答：主要看调试的时候是否振荡。

问：汇编写的函数在 C 语言名字就是地址，在汇编中要加个横岗是什么意思？

答：这是编译器规则要求的，同一个函数名称在汇编中需要加下划线。

那么，下面的话，就要计算积分项的输出。

```
_Ipi_j2:
    clr     w2  ;清零w2寄存器
    cpsgt   w3,w2   ;如果比例项输出w3大于0，跳过下一条语句
    goto    _Ipi_j3 ;如果比例项输出w3小于等于0，跳转到标号_Ipi_j3执行
    mov     w3,w2   ;比例项输出临时存放于w2
    goto    _Ipi_j4
_Ipi_j3:
    neg     w3,w2   ;比例项输出临时存放于w2,符号做了调整
_Ipi_j4:
```

我们来看这段代码啊。下面要做的事情是处理比例项的值，获得它的绝对值。这样的话，方便后面使用来求解积分项的限幅值。实现的思路，其实就是判断比例项输出的正负；正值的话，就不用管，负值的话，就求它的相反数；因为输出值是在正负变化的。

```
_Ipi_j2:
    clr     w2  ;清零w2寄存器
    cpsgt   w3,w2   ;如果比例项输出w3大于0，跳过下一条语句
    goto    _Ipi_j3 ;如果比例项输出w3小于等于0，跳转到标号_Ipi_j3执行
    mov     w3,w2   ;比例项输出临时存放于w2
    goto    _Ipi_j4
_Ipi_j3:
    neg     w3,w2   ;比例项输出临时存放于w2,符号做了调整
_Ipi_j4:
```

判断正负的话，其实就是用比例项输出跟 0 去比较。所以，代码这里先使用 clr　对 w2 寄存器进行清 0，然后，w3 寄存器存放的是比例项的输出值。所以，比较的话就是　w2 跟 w3 去比较。

CPSEQ	Wb, Wn	比较 (Wb – Wn)，如果相等则跳过
CPSGT	Wb, Wn	有符号比较 (Wb – Wn)，如果大于则跳过

这里比较的话，用的是用有符号的比较。大家看，如果 W3 大于 0，就执行红色箭头对应的代码；此时呢，说明 W3 是正值，就把这个值存放到 w2 寄存器了。如果是负值的话，就执行蓝色箭头的代码，此时就要取 W3 的相反数，把负数变为正数。两种情况都判断完毕之后，

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

处理好的比例项输出　正值就存放到了 w2 寄存器中。然后，程序流就会跳转到_Ipi_j4　标号。

```
_Ipi_j3:
    neg     w3,w2    ;比例项输出临时存放于w2,符号做了调整
_Ipi_j4:
    sub     w1,w2,w1     ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A     ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL     ;积分误差累加低16位存入累加器A的低16位
    mov     [w0+pi_Ki],w4 ;积分系数Ki存入w4寄存器
    mov     w3,w5        ;取比例项输出存放在w5寄存器
    mac     w4*w5,A      ;A = A + Ki*误差
    mov     ACCAH,w2     ;取积分累加项高16位存放w2
    cpslt   w2,w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_a ;如果w2>=w1,跳转到_Ipi_sum_a执行
    neg     w1,w1    ;积分限幅值变为负的积分限幅值
    cpslt   w2,w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_ok ;如果w2大于等于w1，跳转到_Ipi_sum_ok执行
_Ipi_sum_a:
    lac     w1,A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
```

下面这段代码，就开始求解积分累加项了。

```
    neg     w3,w2    ;比例项输出临时存放于w2,符号做了调整
_Ipi_j4:
    sub     w1,w2,w1     ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A     ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL     ;积分误差累加低16位存入累加器A的低16位
    mov     [w0+pi_Ki],w4 ;积分系数Ki存入w4寄存器
    mov     w3,w5        ;取比例项输出存放在w5寄存器
    mac     w4*w5,A      ;A = A + Ki*误差
```

首先呢，我们把积分项的限幅值求出来。求解的方法就是总的限幅值输出减去处理好之后的比例项输出。

```
;函数输入：w0
_IpiControl:
    mov     [w0+pi_Kp],w4    ;Kp存放在寄存器w4中
    mov     [w0+pi_sErr],w5 ;误差存放在寄存器w5中
    mpy     w4*w5,A ;Kp*sErr,结果存入累加器A
    sac.r   A,w3     ;比例项输出存放w3
    mov     [w0+pi_sLimit],w1 ;输出限幅值，存在w1寄存器
    cpsgt   w3,w1    ;有符号比较w3和w1，如果w3大于w1，跳过下一条指令
```

之前我们也是把这个输出限幅值存在了 w1 寄存器中。

```
_Ipi_j4:
    sub     w1,w2,w1     ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A     ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL     ;积分误差累加低16位存入累加器A的低16位
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

所以，这个减法减完之后，结果就存在 w1 中，也就是说，w1 寄存器中，就对应积分项的限幅值。

```
_Ipi_j4:
    sub     w1,w2,w1      ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A      ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL      ;积分误差累加低16位存入累加器A的低16位
    mov     [w0+pi_Ki],w4 ;积分系数Ki存入w4寄存器
    mov     w3,w5         ;取比例项输出存放在w5寄存器
    mac     w4*w5,A       ;A = A + Ki*误差
```

因为我们要求解积分累加项，所以呢，每次计算的时候，都要把之前的累加值给取出来。累加值的话，就是存放在 pi_qSumErr32 中的。因为是 32 位，所以需要分别取，先把 32 位的高 16 位取出来，存放到 w2 中，然后，把 w2 中的值装载到累加器 A 中。接着，把 32 位的低 16 位取出来，存放到 w2 中。然后存到累加器 A 的低 16 位。

```
_Ipi_j4:
    sub     w1,w2,w1      ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A      ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL      ;积分误差累加低16位存入累加器A的低16位
```

这里，新接触到一条指令 lac，它就是把 w 寄存器中的值，装载到累加器 A 中的 bit16~bit31。因为积分项要累加，所以也就使用到累加器 A，所以，需要把上一次的累加值存到累加器中。

```
_Ipi_j4:
    sub     w1,w2,w1      ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2 ;积分误差累加高16位存放w2寄存器
    lac     w2,A      ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2 ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL      ;积分误差累加低16位存入累加器A的低16位
    mov     [w0+pi_Ki],w4 ;积分系数Ki存入w4寄存器
    mov     w3,w5         ;取比例项输出存放在w5寄存器
    mac     w4*w5,A       ;A = A + Ki*误差
    mov     ACCAH,w2      ;取积分累加项高16位存放w2
    cpslt   w2,w1     ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_a  ;如果w2>=w1,跳转到_Ipi_sum_a执行
    neg     w1,w1     ;积分限幅值变为负的积分限幅值
    cpslt   w2,w1     ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_ok ;如果w2大于等于w1，跳转到_Ipi_sum_ok执行
_Ipi_sum_a:
```

接着呢，就来计算当前的积分项，也就是使用 Ki*Kp*Err。根据我们之前给出来的公式，也就对应积分系数 Ki 乘以比例项输出。而我们会使用到一条 DSP　相乘并累加指令。所以，需要把两个乘数放到支持的寄存器中，对应的就是 w4w5。接着呢，就是用 mac　相乘并累加

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

指令，计算积分项的累加和。累加的结果，就是存放到累加器 A 中的。这条 mac 指令，我们来看下。

表 3-11： DSP 类指令（dsPIC30F、dsPIC33F 和 dsPIC33E 器件）

汇编语法		说明	字数	周期数	页码
ADD	Acc	累加器相加	1	1	103
ADD	Wso,#Slit4,Acc	16 位有符号数与累加器相加	1	1(1)	104
CLR	Acc,[Wx],Wxd,[Wy],Wyd,AWB	清零累加器	1	1	186
ED	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离（无累加）	1	1	239
EDAC	Wm*Wm,Acc,[Wx],[Wy],Wxd	求取欧几里德距离	1	1	241
LAC	Wso,#Slit4,Acc	装载累加器	1	1(1)	265
MAC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并累加	1	1	275
MAC	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	平方并累加	1	1	277
MOVSAC	Acc,[Wx],Wxd,[Wy],Wyd,AWB	将 Wx 中内容传送到 Wxd，并将 Wy 中内容传送到 Wyd	1	1	293
MPY	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd	Wn 和 Wm 相乘，结果存入累加器	1	1	295
MPY	Wm*Wm,Acc,[Wx],Wxd,[Wy],Wyd	求平方，结果存入累加器	1	1	297

大家看下，前面讲到的 LAC 指令　也属于 DSP 指令，它的功能就是把 w 工作寄存器中的值装载到累加器中。

然后　MAC 指令的话，是相乘并累加，乘法的话，跟我们之前讲的 MPY 指令作用是一样的，就是计算两个 Q15 的乘积。但是跟 MPY 不同的是 MAC 还带了累加的功能。

语法：　{标号:} MAC　　Wm*Wn, Acc {,[Wx], Wxd}　　{,[Wy],Wyd}　{,AWB}
{,[Wx] + = kx, Wxd}　　{,[Wy] + = ky, Wyd}
{,[Wx] – = kx, Wxd}　　{,[Wy] – = ky, Wyd}
{,[W9 + W12],Wxd}　　{,[W11 + W12],Wyd}

操作数：
Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7]
Acc ∈ [A,B]
Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7]
Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7]
AWB ∈ [W13, [W13] + = 2]

操作：
(Acc(A 或 B)) + (Wm) * (Wn) → Acc(A 或 B)
([Wx]) → Wxd； (Wx) + kx → Wx
([Wy]) → Wyd； (Wy) + ky → Wy
舍入后的 (Acc(B 或 A)) → AWB

受影响的状态位：OA、OB、OAB、SA、SB 和 SAB

指令编码：

1100	0mmm	A0xx	yyii	iijj	jjaa

描述：将两个工作寄存器的内容相乘，可选操作包括预取操作数以为另一条 MAC 类

大家看指令描述这个指令描述，就是累加器加上乘积，结果还放在累加器中。后面的部分，打括号的，是省略的。

```
    neg     w3,w2    ;比例项输出临时存放于w2,符号做了调整
_Ipi_j4:
    sub     w1,w2,w1     ;积分项限幅值等于总输出限幅值减去比例项输出，w1作为积分项限幅值使用
    mov     [w0+pi_qSumErr32_H],w2  ;积分误差累加高16位存放w2寄存器
    lac     w2,A     ;积分误差累加高16位存入累加器A的高16位
    mov     [w0+pi_qSumErr32_L],w2  ;积分误差累加低16位存放w2寄存器
    mov     w2,ACCAL     ;积分误差累加低16位存入累加器A的低16位
    mov     [w0+pi_Ki],w4  ;积分系数Ki存入w4寄存器
    mov     w3,w5        ;取比例项输出存放在w5寄存器
    mac     w4*w5,A      ;A = A + Ki*误差
    mov     ACCAH,w2     ;取积分累加项高16位存放w2
    cpslt   w2,w1    ;如果w2小于w1，跳过下一条指令
```

所以，这段代码的话，就是实现积分项计算并累加。

问：为什么积分项的限幅值是总的限幅值减去比例项的呢？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：这里把限幅值分为比例项限幅值和积分项限幅值两部分。限幅值 = 比例项限幅值 + 积分项限幅值。其中，比例项限幅值分配优先级高，也就是说当偏差较大时，比例项占比较高，积分项占比小。当偏差较小时，比例项占比较小，积分项占比大。

问：是不是启动函数必须要这些都要执行完，采取执行 C 函数代码？你这里的顺序是一直顺序执行？这个名字是随便起的吗？

答：当然，启动函数执行完，才会跳转到主程序。汇编标号的名称是程序员自己定义的。

问：这个什么意思 [w0+pi_Kp]？

答：中括号中的是地址，整体代表这个地址中的数据。

$$U(n)=Kp*error(n)+Ki*\sum_{i=0}^{n}Kp*error(i)$$

这个公式前面用过很长时间讲过，是串联式绝对型 PI，大家课后可以再看一下前面讲过的内容。我们继续往下讲了哈。累加项求出来了，还要对累加器的结果做一个判断，看看是不是大于我们设定的限幅值了？

```
mov     w3, w5          ;取比例项输出存放在w5寄存器
mac     w4*w5, A        ;A = A + Ki*误差
mov     ACCAH, w2       ;取积分累加项高16位存放w2
cpslt   w2, w1    ;如果w2小于w1，跳过下一条指令
goto    _Ipi_sum_a  ;如果w2>=w1，跳转到_Ipi_sum_a执行
neg     w1, w1    ;积分限幅值变为负的积分限幅值
cpslt   w2, w1    ;如果w2小于w1，跳过下一条指令
goto    _Ipi_sum_ok ;如果w2大于等于w1，跳转到_Ipi_sum_ok执行
_Ipi_sum_a:
    lac     w1, A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
```

这里的话，我们判断 Q15 的结果就行，所以，就取出累加器的 bit16~bit31 位，临时存放到 w2 中。然后，积分项的限幅值之前求取出来之后，是放到 w1 寄存器中的。所以呢，也就是用 w1 跟 w2 判断。

```
    mov     ACCAH, w2       ;取积分累加项高16位存放w2
    cpslt   w2, w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_a  ;如果w2>=w1，跳转到_Ipi_sum_a执行
    neg     w1, w1    ;积分限幅值变为负的积分限幅值
    cpslt   w2, w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_ok ;如果w2大于等于w1，跳转到_Ipi_sum_ok执行
_Ipi_sum_a:
    lac     w1, A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看下，如果说 W2 小于 W1，也就说明积分累加值小于正的积分限幅值。此时呢，就是执行第一个红色箭头。接着，还要判断积分累加值是不是小于负的积分限幅值。所以，要把积分限幅值取相反数，然后再次判断累加值跟积分限幅值大小。

问：什么情况下会超出限幅值？
答：偏差比较大的情况下。

如果累加值大于负的积分限幅值，就会执行第二个红色箭头。第二个箭头是一条跳转指令，就跳转到标号_Ipi_sum_ok 处。这是累加值都在正负限幅值以内的情况。也就是说比例项至限幅剩余部分，才是留给积分项的范围。否则，可能积分会饱和。

```
    mov     ACCAH,w2     ;取积分累加项高16位存放w2
    cpslt   w2,w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_a  ;如果w2>=w1,跳转到_Ipi_sum_a执行
    neg     w1,w1    ;积分限幅值变为负的积分限幅值
    cpslt   w2,w1    ;如果w2小于w1，跳过下一条指令
    goto    _Ipi_sum_ok ;如果w2大于等于w1，跳转到_Ipi_sum_ok执行
_Ipi_sum_a:
    lac     w1,A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
```

那么，如果累加值大于正的限幅值　或者小于负的限幅值，都会跳转到标号_Ipi_sum_a 处。

这里呢，就是把积分限幅值给装载到累加器的 bit16~bit31 中。这样的话，也就实现了积分累加求解及限幅处理。　高 16 位理解为整数部分，低 16 位理解为小数部分。

```
_Ipi_sum_a:
    lac     w1,A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
    mov     ACCAH,w1     ;累加器高16位存入w1
    mov     w1,[w0+pi_qSumErr32_H] ;w1作为积分误差累加高16位
    mov     ACCAL,w2     ;累加器高16位存入w2
    mov     w2,[w0+pi_qSumErr32_L] ;w2作为积分误差累加低16位
    add     w3,w1,w3     ;比例项输出加上积分项输出，得到Sout
    mov     w3,[w0+pi_sOut] ;比例积分项输出相加，给pi_sOut，得到PI控制器输出
    return
/*=============================================================================*
```

然后，接着这四句话的目的是要更新累加变量。所以，需要把累加器的高 16 位，低 16 位都取出来，然后放到对应的变量中，以便于下一个计算累加项。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
_Ipi_sum_a:
    lac     w1,A     ;如果积分累加项高16位小于负积分限幅值，将积分限幅值存入A
_Ipi_sum_ok:
    mov     ACCAH,w1     ;累加器高16位存入w1
    mov     w1,[w0+pi_qSumErr32_H] ;w1作为积分误差累加高16位
    mov     ACCAL,w2     ;累加器高16位存入w2
    mov     w2,[w0+pi_qSumErr32_L] ;w2作为积分误差累加低16位
    add     w3,w1,w3     ;比例项输出加上积分项输出，得到Sout
    mov     w3,[w0+pi_sOut] ;比例积分项输出相加，给pi_sOut，得到PI控制器输出
    return
/*=============================================================================*/
```

然后大家看，把比例项输出加上积分项的输出，对应的 PI 控制器输出也就得到了。

问：这里加个限幅有什么用处啊？

答：限幅是为了防止过调整。还有防止比例项和积分项之和大于 1，因为整个过程都是归一化处理的。

问：意思是这两个限幅最大不能超过 1？

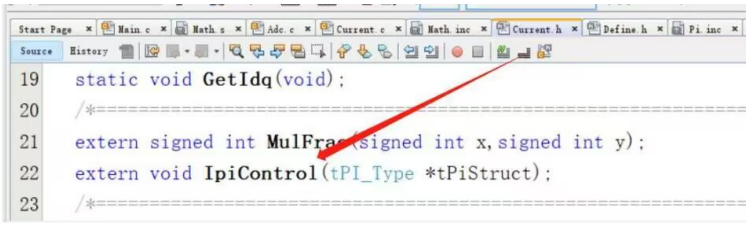
答：是的，PI 的输出项，也是归一化数据。

这样的话，PI 控制的汇编代码就分析完了。

```
;=============================================================================
    .include    "../Include/p33EP32MC204.inc"

    .global     _IpiControl ;函数全局调用声明
/*=============================================================================*/
```

这个函数要调用的话，就要全局声明。



然后 C 文件中的声明是这样的。声明好之后呢，C 语言就可以调用它了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void CurrentLoop(void)
{
    GetIdq();

    tIm.qIdGiven = 0;
    tPiId.sErr = tIm.qIdGiven - tIm.qId;
    tPiId.Kp = 8192;
    tPiId.Ki = tPiId.Kp >> 6;
    tPiId.sLimit = 32767;
    IpiControl(&tPiId);
    tUm.qUd = tPiId.sOut;

    tPiIq.sErr = tIm.qIqGiven - tIm.qIq;
    tPiIq.Kp = 8192;
    tPiIq.Ki = tPiId.Kp >> 6;
    tPiIq.sLimit = 32767;
    IpiControl(&tPiIq);
    tUm.qUq = tPiIq.sOut;
}
```

这个就是我们调用的方式，实现电流闭环控制。

```
/*==========================================================
void CurrentLoop(void)
{
    GetIdq();

    tIm.qIdGiven = 0;
    tPiId.sErr = tIm.qIdGiven - tIm.qId;
    tPiId.Kp = 8192;
    tPiId.Ki = tPiId.Kp >> 6;
    tPiId.sLimit = 32767;
    IpiControl(&tPiId);
    tUm.qUd = tPiId.sOut;
```

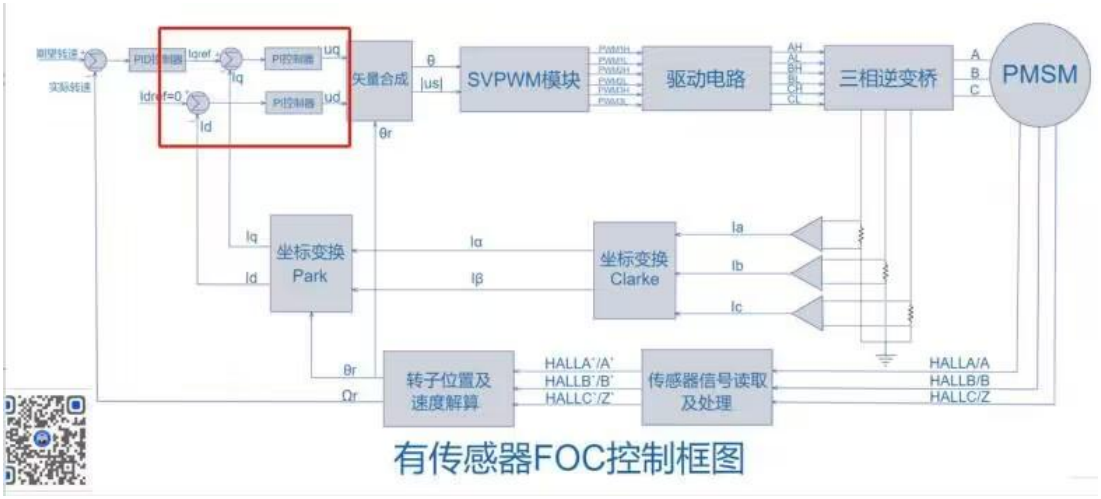
大家看下，红框部分都是设定计算需要的值。例如，电流环的给定，计算误差，设定 Kp，Ki 系数。控制环执行之后呢，就会得到对应的输出值。对 Id 电流控制，得到的就是 Ud。这个值的话，会参与电压矢量模长跟角度的计算。当然，实际的 Kp，Ki 参数可能需要调整。我们后面会专门讲参数的整定。

```
tPiIq.sErr = tIm.qIqGiven - tIm.qIq;
tPiIq.Kp = 8192;
tPiIq.Ki = tPiId.Kp >> 6;
tPiIq.sLimit = 32767;
IpiControl(&tPiIq);
tUm.qUq = tPiIq.sOut;
```

然后，同样的，对 Iq 进行控制，就得到 Uq。这个 Uq 呢，也会参与电压矢量以及角度的计算。

问：kp ki 为什么是这个值随意给吗？
答：不能。后面会专门讲参数的整定。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



所以大家再来看这幅图，就是今天内容的位置。
大家课后消化一下内容，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。